



FAKULTI SAINS KOMPUTER DAN MATEMATIK (FSKM)

SEMESTER II 2025/2026

CSE3433 SOFTWARE ARCHITECTURE (K2)

Deliverable 4: Project Report & Individual Logbook

GROUP 8

Service-Oriented Smart Campus Support System

(Service-Oriented Architecture)

LECTURER:

Dr. MOHAMAD NOR BIN HASSAN

GROUP MEMBERS:

NAME	NO. MATRIC
NOR SAERAH BINTI ISMAIL	S75325
NOR AIN NATASYA BINTI KAMARULZAMAN	S76002
CHE KU NURULHUDA BINTI CHE KU SAIFUL MUZAHAR	S75871

Table Of Content

Table Of Content.....	0
List Of Tables.....	3
List Of Figures.....	4
Figure 7.14: Notification Servlet Implementation.....	4
1.0 Introduction.....	5
2.0 Problem Description.....	5
3.0 System Requirements.....	6
3.1 Functional Requirements.....	6
3.1.1 Identity and Access Management.....	6
3.1.2 Academic and Course Management.....	6
3.1.3 Facility Booking.....	7
3.1.4 Campus Support and Helpdesk.....	7
3.1.5 Notification.....	8
3.2 Non-Functional Requirements.....	8
3.2.1 Performance.....	8
3.2.2 Security.....	8
3.2.3 Usability.....	8
3.2.4 Reliability & Maintainability.....	8
3.2.5 Availability.....	9
3.2.6 Scalability.....	9
4.0 Architecture Design.....	9
4.1 Key Design Principles.....	9
4.2 Service Classifications.....	10
5.0 Architecture Diagrams.....	10
5.1 System Context Diagram.....	10
5.2 Service Architecture Diagram.....	11
5.3 Interaction Diagram.....	12
5.4 Deployment Diagram.....	15
6.0 Technology Stack.....	17
7.0 Prototype Implementation.....	19
7.1 GitHub Repository Information.....	19
7.2 Define Interfaces/APIs/Events.....	19
8.0 Architecture Evaluation.....	30
8.1 Quality Attribute Evaluation.....	30
8.2 SOA Principle Compliance.....	30
8.3 Architecture Tradeoffs.....	31
8.4 Comparison with Alternative Architectures.....	32
9.0 Team Contributions.....	33
10.0 Conclusion and Reflection.....	34

10.1 Project Conclusion.....	34
10.2 What We Learned.....	34
10.3 Future Improvements:.....	35

List Of Tables

Table 6.1: Technologies and tools for The Smart Campus Support System.....	14
Table 7.1.1: GitHub Repository Information.....	16
Table 7.2.1: Interface for Request Gateway Service (RouterServlet.java).....	17
Table 7.2.2: Interface 2 for IT Ticketing Service (ITDepartmentServlet.java).....	19
Table 7.2.3: Interface 3 for Facility Ticketing Service (FacilityDepartmentServlet.java)	22
Table 7.2.4: Interface 4 for Notification Service (NotificationServlet.java).....	25
Table 8.1: Quality Attribute Evaluation.....	27
Table 8.2: SOA Principle Compliance.....	27
Table 8.3: Architecture Tradeoffs.....	28
Table 8.4: Comparison with Alternative Architectures.....	28
Table 9.1: Team Contributions.....	30

List Of Figures

Figure 5.1: System Context Diagram for Service-Oriented Smart Campus Support System.....	7
Figure 5.2: Service Architecture Diagram for Service-Oriented Smart Campus Support System.....	8
Figure 5.3.1: Interaction Diagram for Issue Reporting.....	10
Figure 5.3.2: Interaction Diagram for Course Registration.....	11
Figure 5.3.3: Interaction Diagram for Facilities Booking.....	11
Figure 5.3.4: Interaction Diagram for Admin Staff Add Facility.....	12
Figure 5.4: Deployment Diagram for Service-Oriented Smart Campus Support System.....	12
Figure 7.1: RouterServlet Code Implementation for Request Gateway Service.....	17
Figure 7.2: Request Routing Logic Based on Department Category.....	18
Figure 7.3: ITDepartmentServlet Code Implementation for IT Ticketing Service.....	19
Figure 7.4: Database Connection and Query Execution for IT Service.....	20
Figure 7.5: IT Ticket Record Creation in MySQL Database.....	20
Figure 7.6: IT Service Request Success Message Rendering.....	20
Figure 7.7: IT Support Notification Displayed in System Console.....	21
Figure 7.8: FacilityDepartmentServlet Code Implementation for Facility Ticketing Service.....	22
Figure 7.9: Database Interaction for Facility Ticket Submission.....	23
Figure 7.10: Facility Ticket Record Successfully Saved in Database.....	23
Figure 7.11: Facility Support Request Success Page.....	23
Figure 7.12: Facility Support Notification Log Output.....	24
Figure 7.13: Console Output for Notification Service.....	25
Figure 7.14: Notification Servlet Implementation.....	25

1.0 Introduction

The Service-Oriented Smart Campus Support System is a centralized digital ecosystem that is designed to replace the divided and disorganised infrastructures that currently make it hard for universities to run smoothly. By using the Service-Oriented Architecture (SOA), the platform will combine all of the isolated functions such as academic registration, resource scheduling and administrative support into a single and interoperable hub.

This transition from isolated platforms to an integrated service model that can eliminate data redundancy, strengthens security through centralized identity management and provides a smooth user experience for both students and faculty.

2.0 Problem Description

Modern university campuses face challenges in managing student services due to fragmented and poorly integrated systems. Students and staff often use multiple platforms for tasks like course registration, facility booking, issue reporting, and receiving updates, which leads to inconvenience, duplicated data, and delays in accessing services.

For instance, course registration may lack real-time validation for prerequisites or seat availability, while facility bookings can cause scheduling conflicts without a centralized system. Issue reporting is also inefficient, with limited visibility on request status. Additionally, the absence of a unified identity and access control system creates security risks and inconsistent authorization across services. Weak communication between systems further results in delayed or missed updates.

To address these issues, a Service-Oriented Smart Campus Support System is proposed. By using Service-Oriented Architecture (SOA), the system integrates key services such as IT Ticketing (Helpdesk), Facility Ticketing (Helpdesk) and Notification into a modular and interoperable structure. This approach improves efficiency, ensures data consistency, enhances user experience and supports future scalability.

3.0 System Requirements

The system requirements include the elements below:

3.1 Functional Requirements

The Service-Oriented Smart Campus Support System is divided into five initial functions. Each function is designed to be autonomous while providing users the specific capabilities via RESTful API endpoints.

3.1.1 Identity and Access Management

This function acts as the important authority for user verification and role management in the system.

- FR-101: The system shall allow students and staff to log in using their unique Username and password.
- FR-102: The service shall provide an endpoint for other services to verify if a user has "Student" or "Admin" privileges.
- FR-103: The system shall provide an endpoint to validate if a user's session is still active before allowing access to other services.
- FR-104: The system shall retrieve user profile data (Name, Student ID, and Role) upon request from authorized services.

3.1.2 Academic and Course Management

This service manages the academic catalog and the enrollment process for students.

- FR-201: The system shall provide a list of available courses, including course codes, names, and credit hours.
- FR-202: The system shall allow students to register for specific courses during the open registration period.
- FR-203: The service shall verify if a student meets the requirements for a course before allowing registration.
- FR-204: The system shall automatically close registration for a course once the maximum number of students is reached.

- FR-205: The service shall verify the student's active status via the Identity function before finalizing any course enrollment.

3.1.3 Facility Booking

This service manages the digital reservation of physical campus resources like discussion rooms and computer labs.

- FR-301: The system shall display a list of available rooms (Discussion Room A, Lab 1) along with their availability status.
- FR-302: Students shall be able to create, view, and cancel reservations for specific time slots.
- FR-303: Conflict Prevention: The system shall validate that a room does not have another booking for the same time period.
- FR-304: The system shall communicate with the Identity Service to verify the student's eligibility before confirming a reservation.

3.1.4 Campus Support and Helpdesk

This function handles the technical and administrative requests and allows students to report issues digitally.

- FR-401: Users shall be able to submit an issues report with a title, description, and priority level (Low, Medium, High).
- FR-402: Students shall be able to track the progress of their issues report("Pending," "In Progress," "Resolved").
- FR-403: Staff shall be able to assign the issues report to specific technicians for resolution.

3.1.5 Notification

This utility function will handle the communication and user feedback.

- FR-501: The system shall trigger an automatic "Booking Successful" alert once a reservation is recorded in the facility booking functions.
- FR-502: The system shall notify the student whenever the status of their support ticket changes (from "Pending" to "Resolved").
- FR-503: The system shall provide a real-time notification to Administrators when a new facility is added or a critical issues report is submitted.
- FR-504: The system shall alert students when the registration deadline is approaching.

3.2 Non-Functional Requirements

This section will outline the quality attributes and some of the operational constraints for the Service-Oriented smart Campus System.

3.2.1 Performance

- NFR-101 : The system shall support at least 5,000 concurrent users with an average response time less than 2 seconds during peak periods such as course registration weeks.

3.2.2 Security

- NFR-201 : The system shall implement Centralized Role-Based Access Control (RBAC) via the Identity Management service to restrict access to sensitive academic and administrative data.

3.2.3 Usability

- NFR-301 : The UI must be fully responsive, ensuring a seamless experience across all of the web browsers, tablets and mobile devices.

3.2.4 Reliability & Maintainability

- NFR-401 : The failure of one non-critical service such as Helpdesk shall not cause the failure of other services like Identity Management.

- NFR-402 : Automated daily backups of all databases must be maintained to ensure zero data loss in the event of a system failure.

3.2.5 Availability

- NFR-501 : The system shall maintain an uptime of 99.9% with 24/7 availability, excluding scheduled maintenance windows.

3.2.6 Scalability

- NFR-601 : The system shall be capable of scaling horizontally to accommodate a growth of up to 10,000 active users as the university expands.

4.0 Architecture Design

This Smart Campus Support System will be designed using the RESTful web services within a Service-Oriented Architecture to break down the systems into four autonomous and loosely coupled services such as RouterServlet (Request Gateway), ITDepartmentServlet, FacilityDepartmentServlet, NotificationServlet

4.1 Key Design Principles

The system follows the Service-Oriented Architecture principles in the following things:

- **Statelessness:** Every time the service gets a request it has all the information it needs to process that request. This will help the system work well with a lot of users and makes it easier to manage the servers
- **Loose Coupling:** The services can work on their own or independently so that the user can update the Facility Booking part without messing with other parts like the Identity or Academic parts because of the minimal dependency it has.
- **Standardized Contracts:** All the services communicate with each other using REST API interfaces and URL-encoded parameters (application/x-www-form-urlencoded). This helps all the different parts of the system work together smoothly.
- **Service Autonomy:** Each service controls its environment, which helps the whole campus system work better.

4.2 Service Classifications

To ensure the business alignments and technical efficiency, the services are categorized as follows:

- **Business Services:** IT Ticketing (ITDepartmentServlet) for issue reporting and Facility Ticketing (FacilityDepartmentServlet) for maintenance requests.
- **Utility Services:** RouterServlet (API Gateway) for request routing and NotificationServlet for email dispatch simulation.
- **Entity Services:** Ticket entities handled by IT and Facility services for storing ticket data in their respective databases.

5.0 Architecture Diagrams

5.1 System Context Diagram

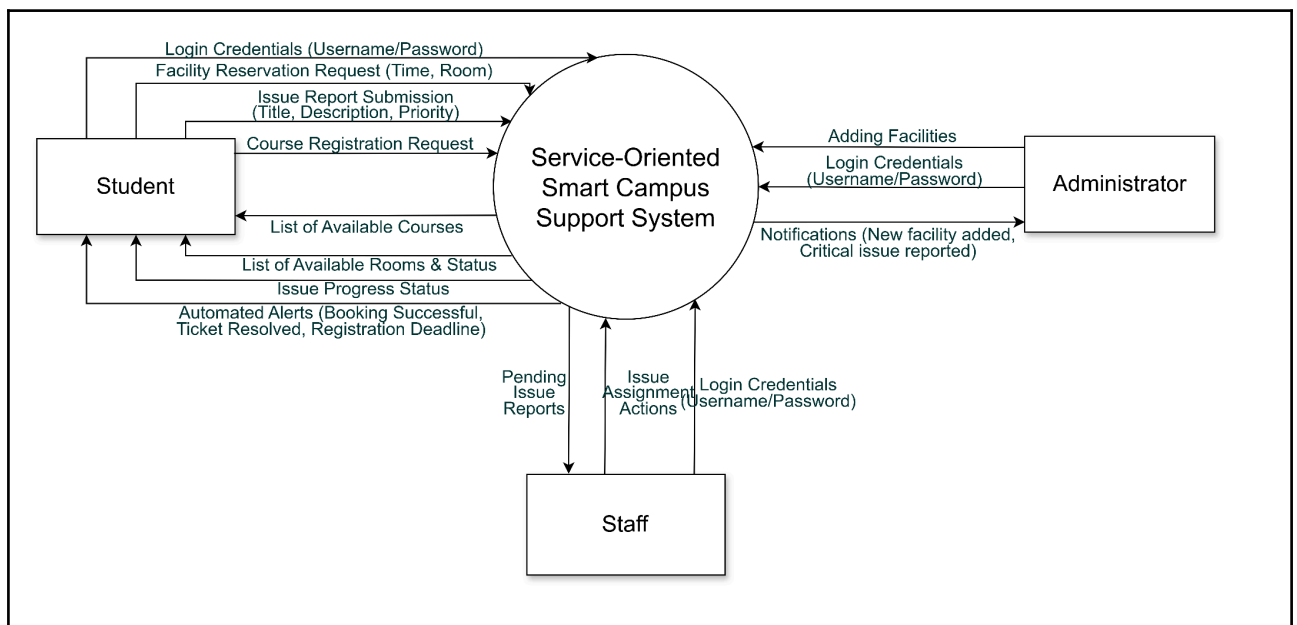


Figure 5.1: System Context Diagram for Service-Oriented Smart Campus Support System

The System Context Diagram in Figure 5.1 above shows the boundary of the Service-Oriented Smart Campus Support System with its context. The system interacts with three primary external entities that are from Students, Staff and Administrator. The internal workings of the system such as the databases and individual services will be hidden in the center which is called "black box" to focus on only what data should go in and what need to come out.

5.2 Service Architecture Diagram

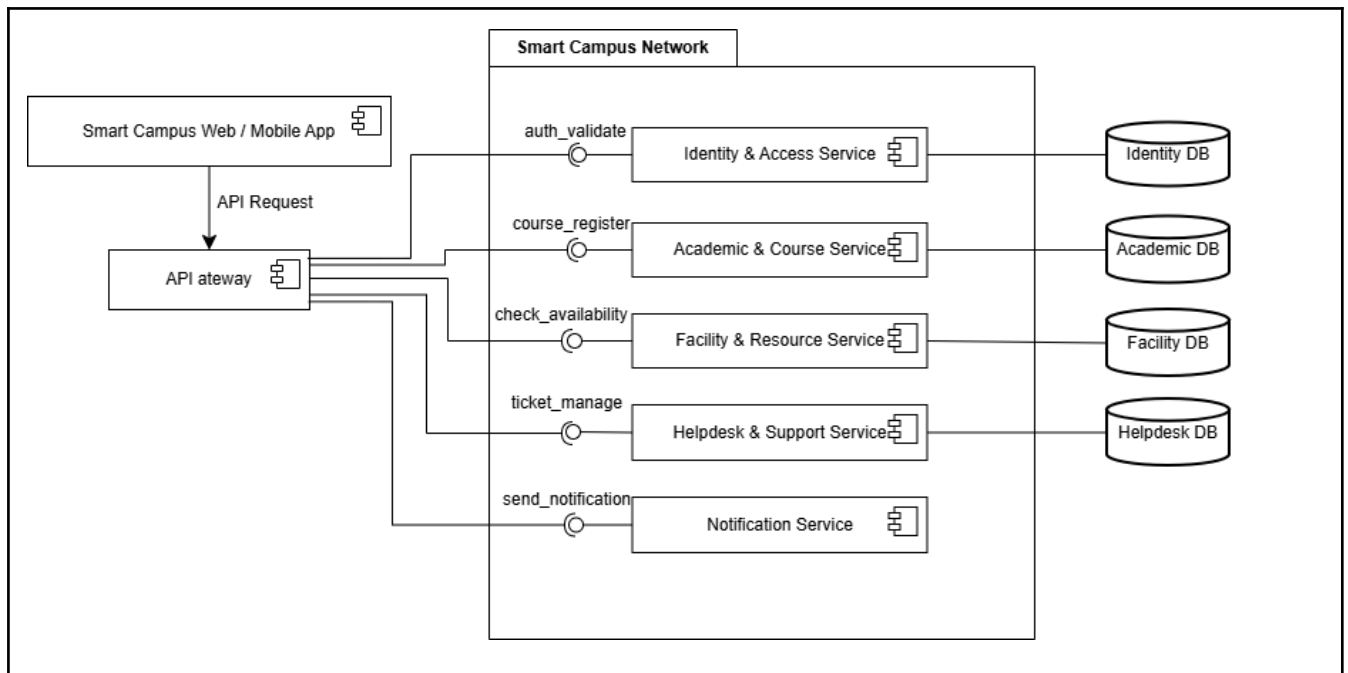


Figure 5.2: *Service Architecture Diagram for Service-Oriented Smart Campus Support System*

The Service Architecture Diagram in Figure 5.2 shows the logic of the Service-Oriented Smart Campus System based on the principles of loose coupling and service autonomy. API Gateway acts as a single mediator that receives requests from client applications and routes traffic to the relevant domain services through predefined standardized interfaces contracts. Each service, such as Identity & Access and Academic & Course, has its own database to ensure Data Ownership principles are followed, while reducing redundancy and improving data security in the campus network. Overall, this design allows each module to function independently while remaining seamlessly integrated through a secure intermediate layer.

5.3 Interaction Diagram

The interaction diagrams show how different services in our Service-Oriented Smart Campus Support System work together for issue reporting. Users submit requests through index.jsp, RouterServlet routes to IT or Facility service and then, ticket saved to database and Notification prints confirmation.

First, users must log in through the Identity & Access Service so the system knows who they are and what they are allowed to do. After logging in, users can do different things based on their roles:

- Students: Can register for courses, book rooms, and report issues.
- Admins: Can manage the system, like adding new facilities.

Each request goes to the right service (like Academic, Facility Booking or Helpdesk). Before saving any data into the database, the system will check if the rules are met and if the data is correct.

During all these processes, the Notification Service is important because it sends real-time updates, like booking confirmations. This shows why Service-Oriented Architecture (SOA) is a good choice. It makes the system faster, prevents duplicate data, and is easier for users to use compared to the old, separated systems.

Key Workflows

- **Scenario 1: The Happy Path (Issue Reporting)**

Client (index.jsp) dispatches an issue to RouterServlet. The router identifies "department": "IT", triggers an outbound HttpURLConnection POST call to ITDepartmentServlet, saves the ticket inside db_it_service, and returns a 200 OK response along with a success message while printing a backend log "IT Service saved a new ticket" to the NetBeans console.

- **Scenario 2: Bad Data (Invalid Category)**

Client dispatches data containing an unknown department value like "HR" or "Unknown"). The validation guard at RouterServlet immediately flags the bad data, stops execution, and returns an error message with "Invalid department. Use 'IT' or 'Facility'".

- **Scenario 3: Fault Tolerance (Service Unavailable)**

Client submits a "Facility" request while the FacilityDepartmentServlet is terminated or unavailable. The RouterServlet intercepts the connection failure via a local try-catch safety net and safely returns an HTTP 503 Service Unavailable with the error "Target service is currently unavailable" without collapsing the primary router application.

5.3.1 Interaction Diagram for Issue Reporting

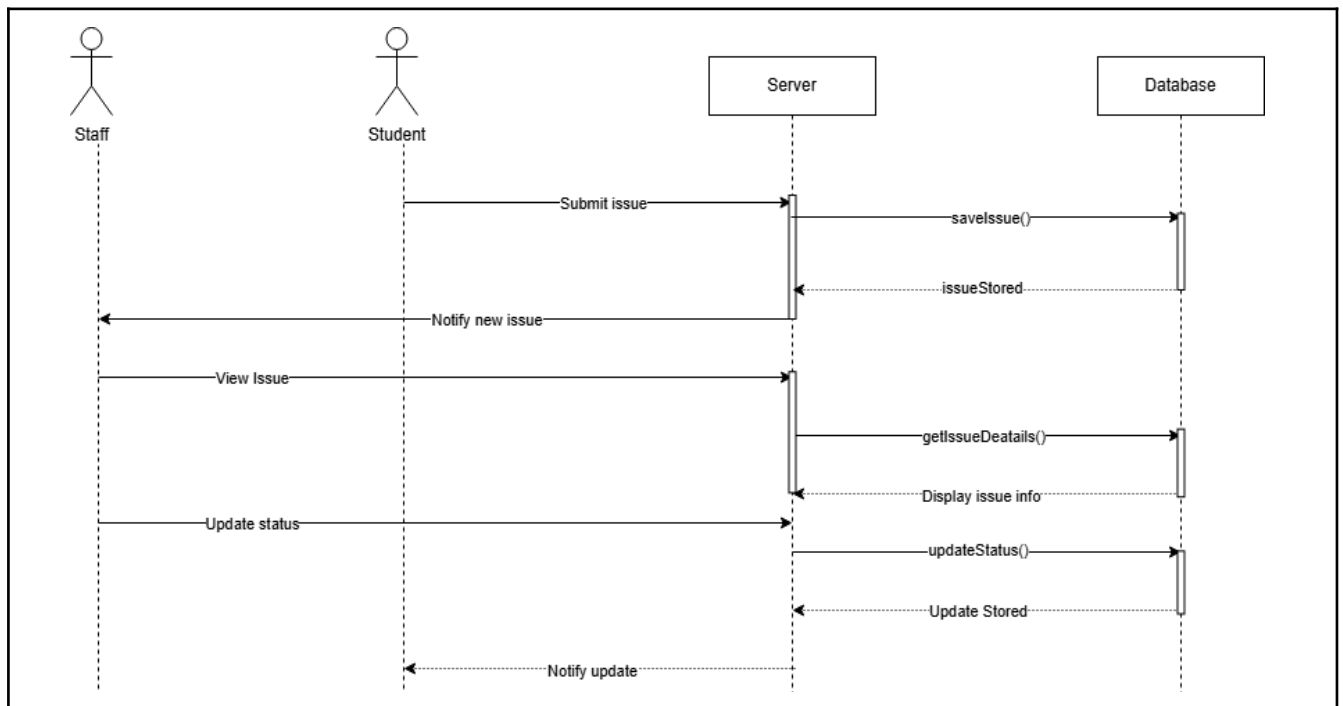


Figure 5.3.1: *Interaction Diagram for Issue Reporting*

5.3.2 Interaction Diagram for Course Registration

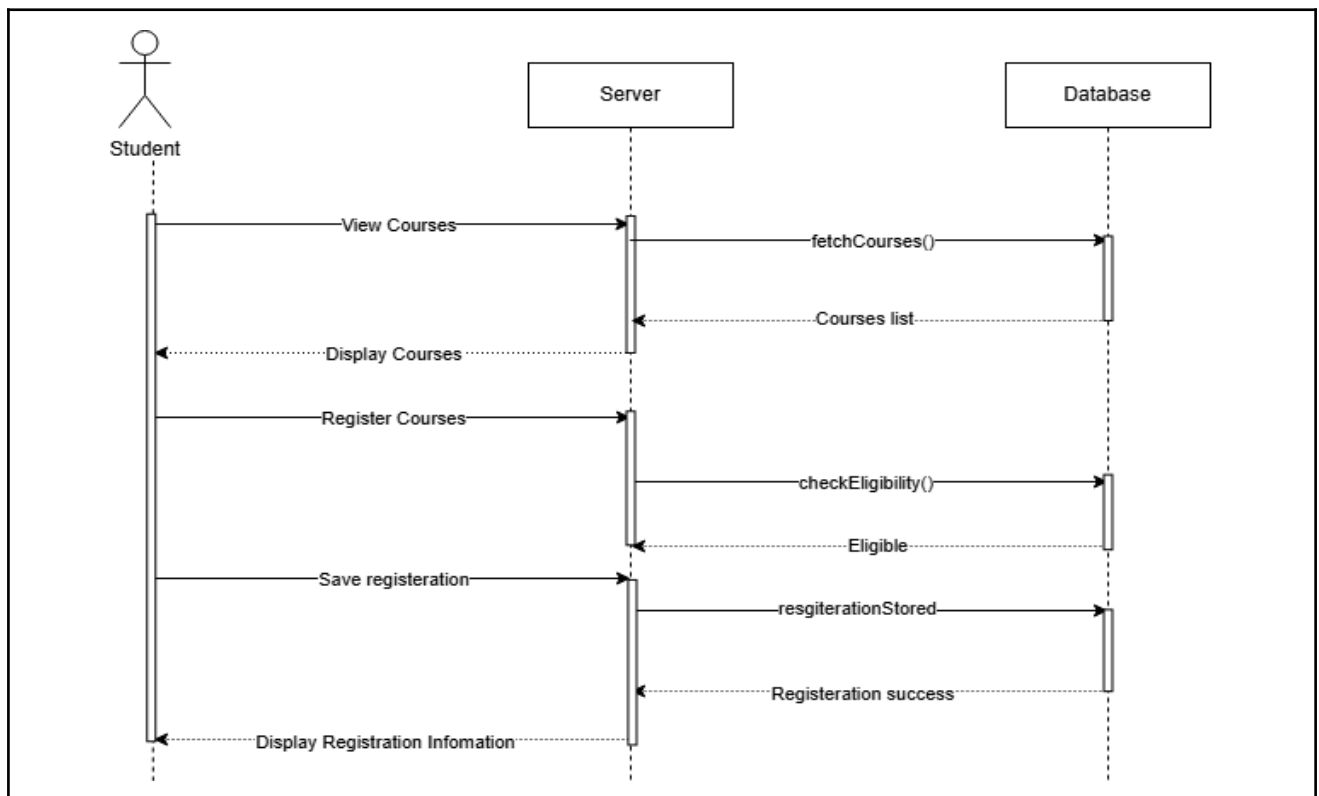


Figure 5.3.2: Interaction Diagram for Course Registration

5.3.3 Interaction Diagram for Facilities Booking

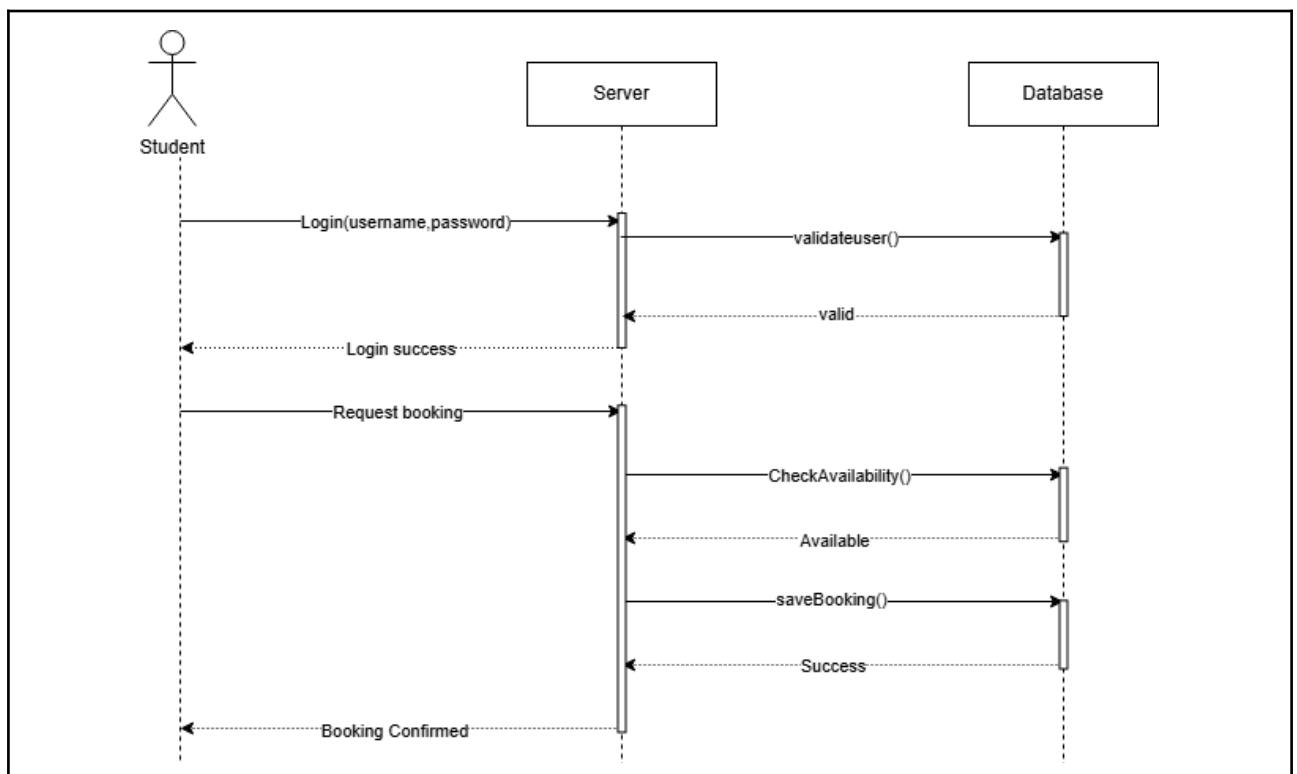


Figure 5.3.3: Interaction Diagram for Facilities Booking

5.3.4 Interaction Diagram for Admin Staff Add Facility

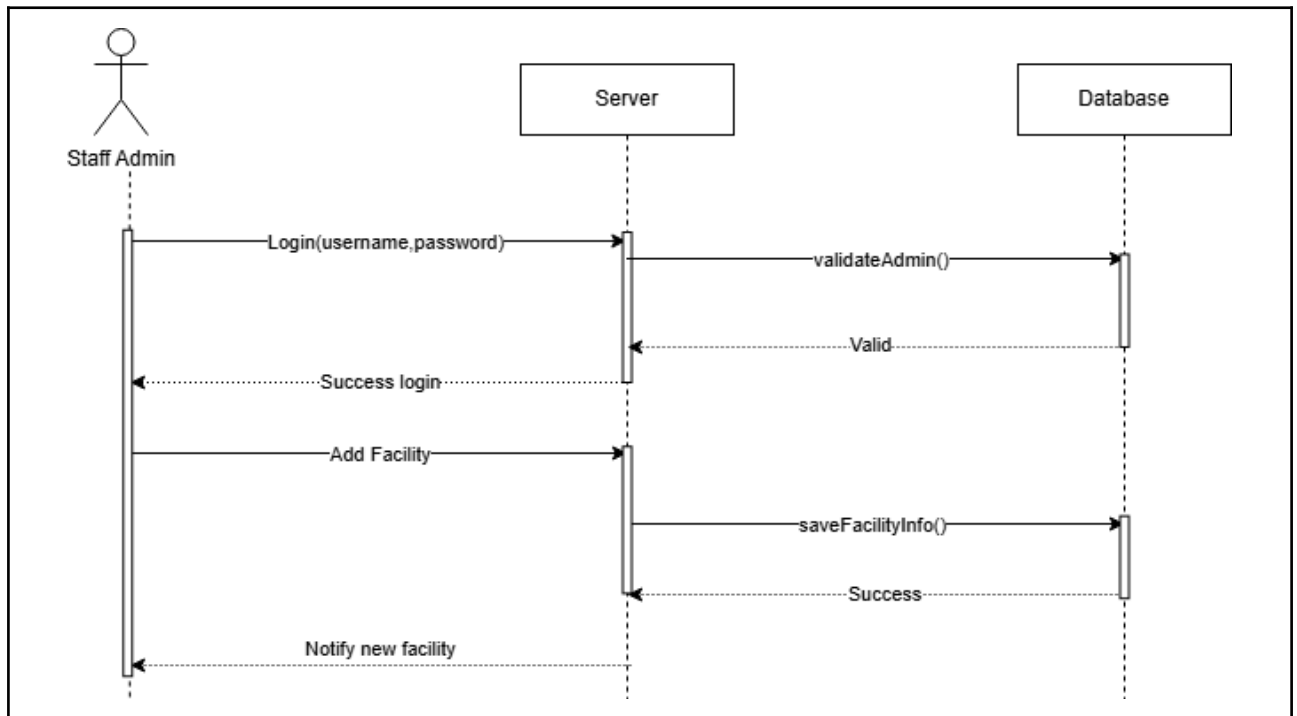


Figure 5.3.4: Interaction Diagram for Admin Staff Add Facility

5.4 Deployment Diagram

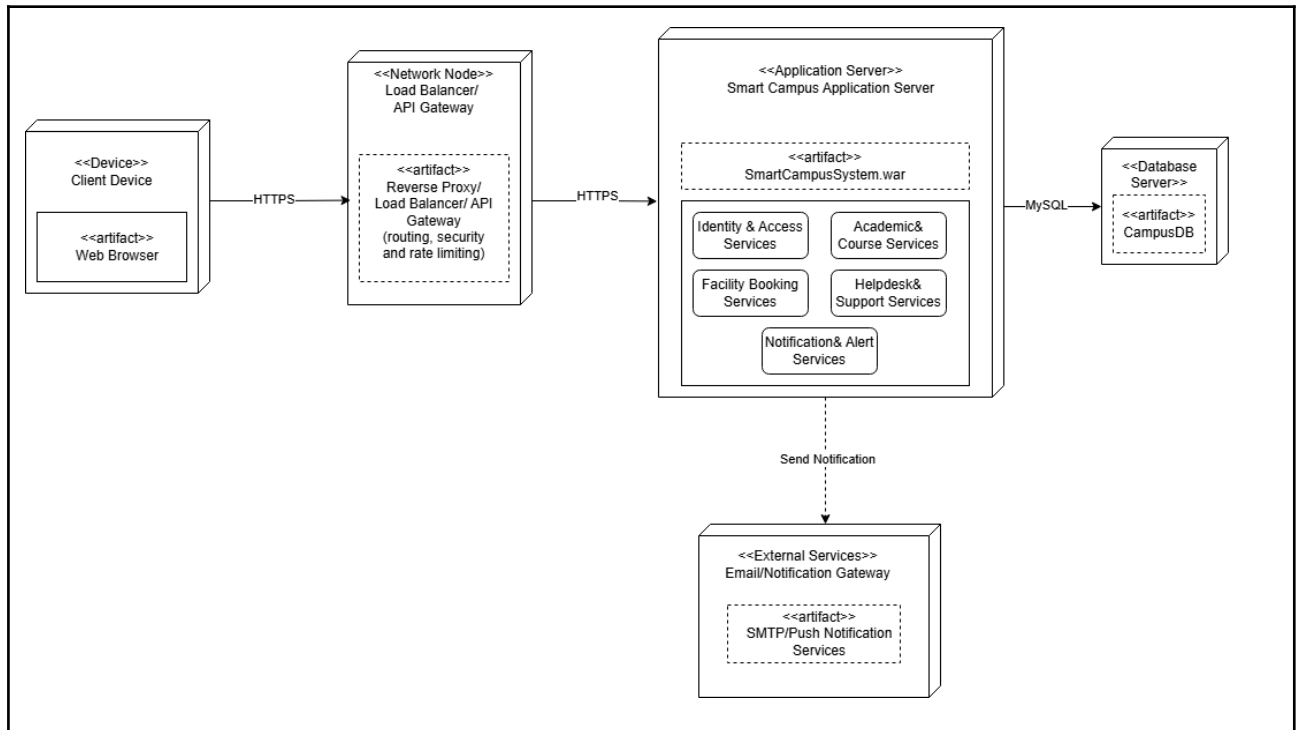


Figure 5.4: Deployment Diagram for Service-Oriented Smart Campus Support System

The diagram shows the architecture of the *Smart Campus Support System*, which follows a service-oriented approach. Users access the system through a web browser and requests are sent via HTTPS to the API Gateway. This component acts as a load balancer and reverse proxy, handling routing, security and rate limiting.

The requests are then processed by the *Smart Campus Application Server*, which contains several service modules such as RouterServlet, ITDepartmentServlet, FacilityDepartmentServlet and NotificationServlet. The server communicates with a MySQL database to store and manage data.

Additionally, the Notification Service connects to an external Email Notification Gateway to send alerts to users. This architecture ensures a secure, scalable and well-organized system.

6.0 Technology Stack

The Smart Campus Support System utilizes the following technologies and tools:

Table 6.1: Technologies and tools for The Smart Campus Support System

Category	Technology	Purpose
Development Environment	NetBeans IDE	Primary IDE for Java web application development, including servlets, JSP pages, and database connectivity configuration
Backend Framework	Java Servlets (javax.servlet)	Implements RESTful endpoints using @WebServlet annotations for handling HTTP requests and routing logic
API Communication	REST APIs over HTTP (POST)	All services communicate via synchronous REST APIs using URL-encoded parameters, ensuring standardized contracts
Database System	MySQL 8.0	Relational database storing ticket data in service-specific databases (db_it_service, db_facility_service)
Database Administration	phpMyAdmin 5.2	Web-based tool for creating databases, tables, and executing SQL queries for data validation
Database Connectivity	MySQL Connector/J (JDBC)	Java JDBC driver enabling database connections via jdbc:mysql://localhost:3306 URLs
Web Frontend	JSP (index.jsp, success.jsp)	HTML-based user interface with embedded Java for form submission and success page rendering
API Gateway	RouterServlet.java	Central entry point validating requests and routing to IT or Facility services based on department field

Notification System	NotificationServlet.java	Stateless service simulating email dispatch by printing confirmation messages to NetBeans console output
Server Runtime	Apache Tomcat 9	Servlet container deployed via NetBeans for running Java Servlets and handling HTTP requests on port 8080
Version Control	GitHub	Source code management with SmartCampusSupportSystem repository containing controller package and web pages
Testing Tools	Postman, Browser	API endpoint testing via Postman and end-to-end workflow testing through web browser at http://localhost:8080

7.0 Prototype Implementation

7.1 GitHub Repository Information

Table 7.1.1: GitHub Repository Information

Attribute	Value
Link	https://github.com/S75871/CSF3433-Group-8-SOA-Smart-Campus-Support-System.git
Repository Name	CSF3433-Group-8-SOA-Smart-Campus-Support-System
Branch	main
Repository Type	Java Web Application (NetBeans)
Package Structure	controller package containing all servlets

Key Implementation Details:

- All servlets use @WebServlet annotations for URL mapping
- Database connections use JDBC with MySQL Connector/J
- Services communicate via HttpURLConnection POST requests
- Each service maintains strict database isolation

7.2 Define Interfaces/APIs/Events

Our system completely rejects direct binary compilation to enforce strict boundaries under Service-Oriented Architecture (SOA). Components communicate purely using Synchronous REST APIs over HTTP protocols, exchanging data via URL-encoded parameters (application/x-www-form-urlencoded).

7.2.1 Interface 1: Request Gateway Service (RouterServlet.java)

Table 7.2.1 : Interface for Request Gateway Service (RouterServlet.java)

Attribute	Value
Endpoint	POST http://localhost:8080/SmartCampusSupportSystem/RouterServlet
HTTP Method	POST
Payload Format	URL-encoded parameters
Payload	form-data: department = Facility issueText = Whiteboard is broken

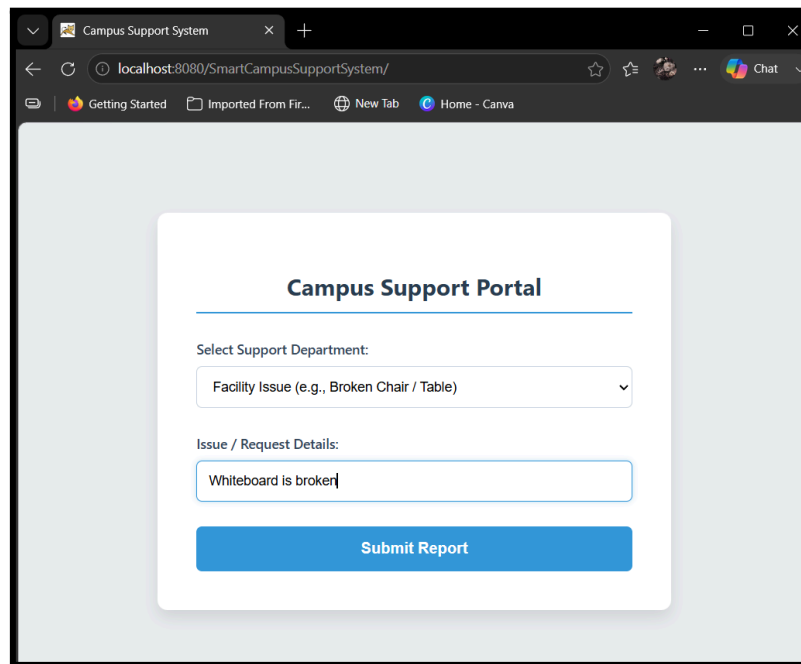
A screenshot of a web browser displaying the 'Campus Support Portal'. The browser's address bar shows 'localhost:8080/SmartCampusSupportSystem/'. The page has a light gray background. In the center, there is a white rectangular form with a blue border. The form is titled 'Campus Support Portal' in bold black text, underlined. Below the title, there is a section labeled 'Select Support Department:' with a dropdown menu showing 'Facility Issue (e.g., Broken Chair / Table)'. Below that, there is a section labeled 'Issue / Request Details:' with a text input field containing 'Whiteboard is broken'. At the bottom of the form, there is a blue button with the text 'Submit Report' in white.

Figure 7.1: RouterServlet Code Implementation for Request Gateway Service

```

String dept = request.getParameter("department");
String issue = request.getParameter("issueText");
String targetURL = "";

// SOA Routing Logic: Avoid one service doing everything
if ("IT".equals(dept)) {
    targetURL = "http://localhost:8080/SmartCampusSupportSystem/ITDepartmentServlet";
} else if ("Facility".equals(dept)) {
    targetURL = "http://localhost:8080/SmartCampusSupportSystem/FacilityDepartmentServlet";
}

try {
    // Forward request to the specific core service (Decomposition)
    URL url = new URL(targetURL);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("POST");
    conn.setDoOutput(true);

    // THE API PAYLOAD (The actual data being sent across the network)
    String postData = "description=" + URLEncoder.encode(issue, "UTF-8");
    try (OutputStream os = conn.getOutputStream()) {
        os.write(postData.getBytes());
        os.flush();
    }
}

```

Figure 7.2: Request Routing Logic Based on Department Category

Explanation:

The Request Gateway Service acts as the central entry point of the Smart Campus Support System. All requests submitted by users are first received by the RouterServlet before being forwarded to any backend service. This service functions as an API Gateway that validates incoming requests and determines the appropriate destination service based on the selected department.

- If department=IT : routes to ITDepartmentServlet
- If department=Facility : routes to FacilityDepartmentServlet

When a user submits a support request through the web interface, the RouterServlet examines the department field contained in the request payload. If the department is identified as "IT", the request is routed to the IT Ticketing Service. If the department is identified as "Facility", the request is forwarded to the Facility Ticketing Service. This routing mechanism ensures that users do not interact directly with backend services or databases.

By centralizing request handling, the RouterServlet promotes loose coupling between services and improves maintainability. Any future service additions or modifications can be managed through the gateway without affecting the client interface.

7.2.2 Interface 2: IT Ticketing Service (ITDepartmentServlet.java)

Table 7.2.2 : Interface 2 for IT Ticketing Service (ITDepartmentServlet.java)

Attribute	Value
Endpoint	POST http://localhost:8080/SmartCampusSupportSystem/ITDepartmentServlet
HTTP Method	POST
Payload Format	URL-encoded parameters
Payload	form-data: department = IT issueText = Broken Projector

The screenshot shows a web browser window with the title 'Campus Support System'. The address bar displays 'localhost:8080/SmartCampusSupportSystem/index.jsp'. The page content features a central white card with the heading 'Campus Support Portal'. Below the heading, there is a section titled 'Select Support Department:' with a dropdown menu showing 'IT Issue (e.g., Broken Projector / Mic)'. Underneath, the 'Issue / Request Details:' section contains a text input field with the value 'Broken Projector'. At the bottom of the card is a blue button labeled 'Submit Report'.

Figure 7.3: *ITDepartmentServlet Code Implementation for IT Ticketing Service*

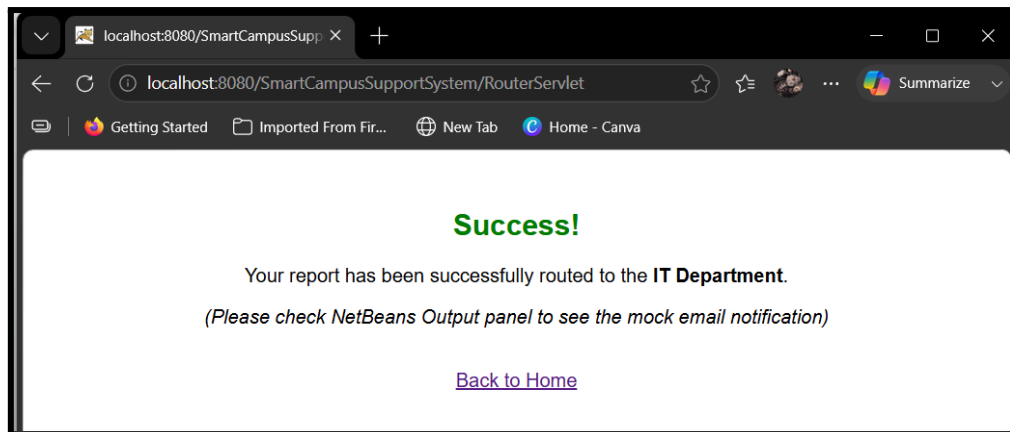


Figure 7.4: Database Connection and Query Execution for IT Service

```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    try (Connection conn = DriverManager.getConnection(dbURL, dbUser, dbPass)) {

        // 1. RESOLVE workflow: Save to IT Database
        String sql = "INSERT INTO it_tickets (description, status) VALUES (?, 'Open')";
        try (PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, description);
            stmt.executeUpdate();
        }
    }
}
```

Figure 7.5: IT Ticket Record Creation in MySQL Database

The screenshot shows the NetBeans Output window with the following text:

```
=====
? [NOTIFICATION SERVICE] Signal Received!
Routed From: IT Department
Status: Dispatching automated email to User/Committee...
Email Body: Your report regarding 'Broken Projector' has been successfully logged into our system.
=====
```

Figure 7.6: IT Service Request Success Message Rendering

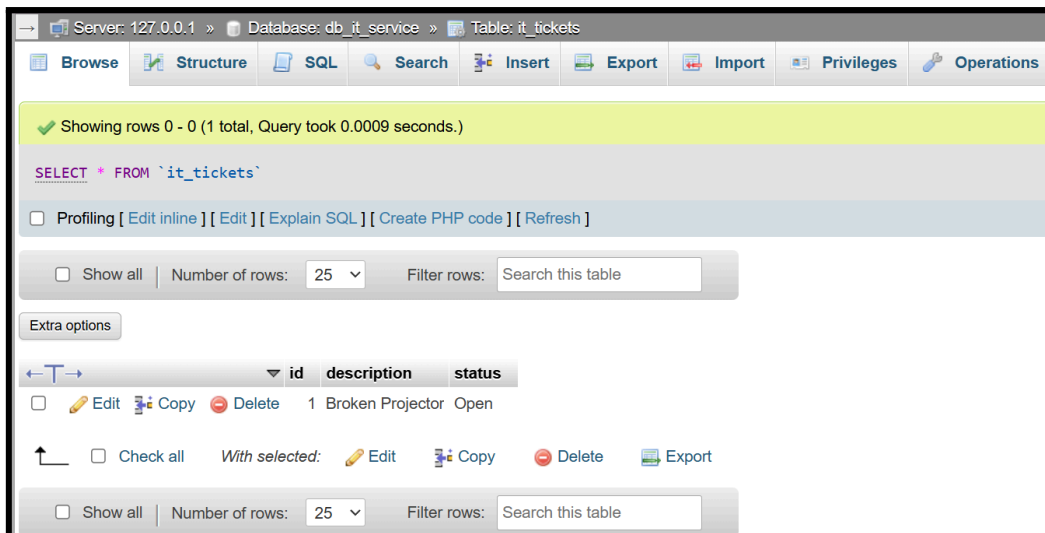


Figure 7.7: *IT Support Notification Displayed in System Console*

Explanation: The IT Ticketing Service is responsible for handling all information technology related support requests submitted by users. Once a request is received from the Request Gateway Service, the ITDepartmentServlet processes the request according to the business rules defined for IT support operations.

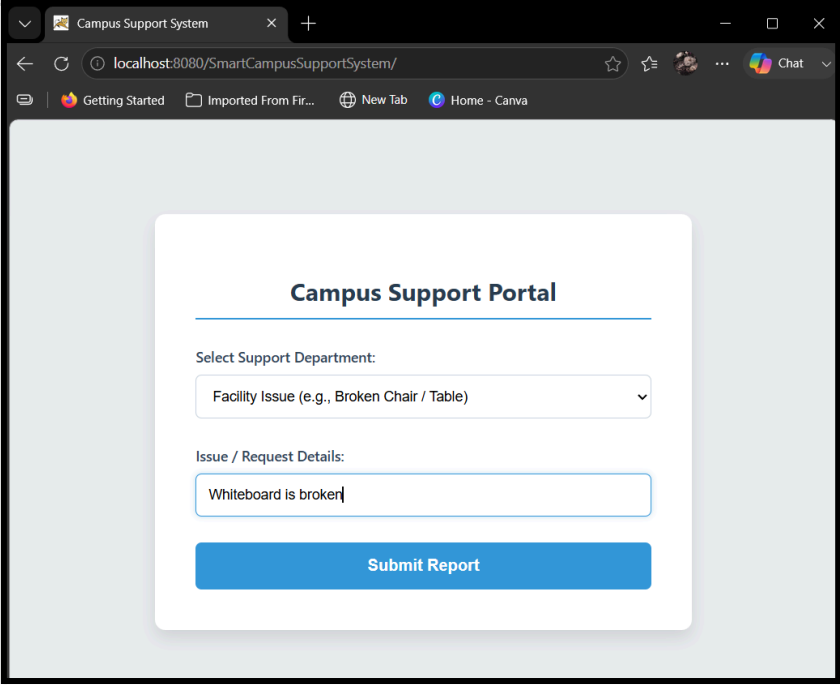
The service validates the received information and stores the ticket details in its dedicated database, db_it_service. This database isolation ensures that IT-related data remains independent from other system services, reducing inter-service dependencies and improving system reliability.

After the ticket has been successfully recorded, the service communicates with the Notification Service to generate a confirmation message indicating that the request has been successfully submitted. This demonstrates service-to-service communication through REST-based interactions, which is a fundamental principle of Service-Oriented Architecture (SOA).

7.2.3 Interface 3: Facility Ticketing Service (FacilityDepartmentServlet.java)

Table 7.2.3: Interface 3 for Facility Ticketing Service (FacilityDepartmentServlet.java)

Attribute	Value
Endpoint	POST http://localhost:8080/SmartCampusSupportSystem/FacilityDepartmentServlet
HTTP Method	POST
Payload Format	URL-encoded parameters
Payload	form-data: department = Facility issueText = Whiteboard is broken



The screenshot shows a web browser window with the title 'Campus Support System'. The address bar shows the URL 'localhost:8080/SmartCampusSupportSystem/'. The page content features a central white card with a light blue border. At the top of the card is the heading 'Campus Support Portal' followed by a horizontal line. Below this, there is a section labeled 'Select Support Department:' with a dropdown menu currently showing 'Facility Issue (e.g., Broken Chair / Table)'. Underneath is a section labeled 'Issue / Request Details:' with a text input field containing the text 'Whiteboard is broken'. At the bottom of the card is a prominent blue button labeled 'Submit Report'.

Figure 7.8: FacilityDepartmentServlet Code Implementation for Facility Ticketing Service

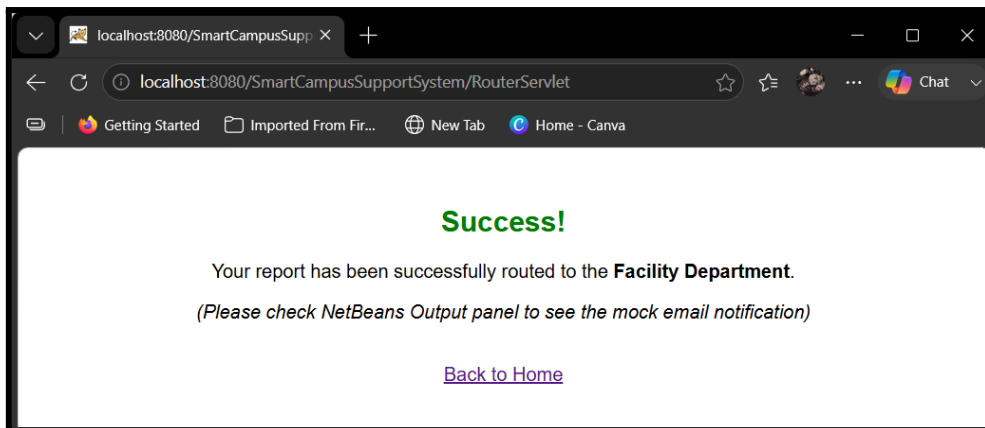


Figure 7.9: Database Interaction for Facility Ticket Submission



Figure 7.10: Facility Ticket Record Successfully Saved in Database

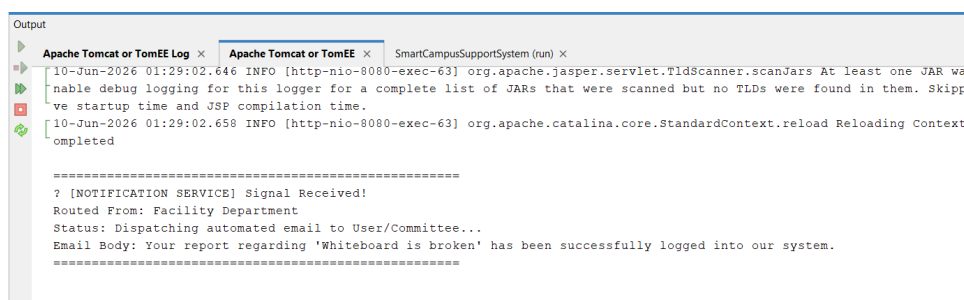


Figure 7.11: Facility Support Request Success Page

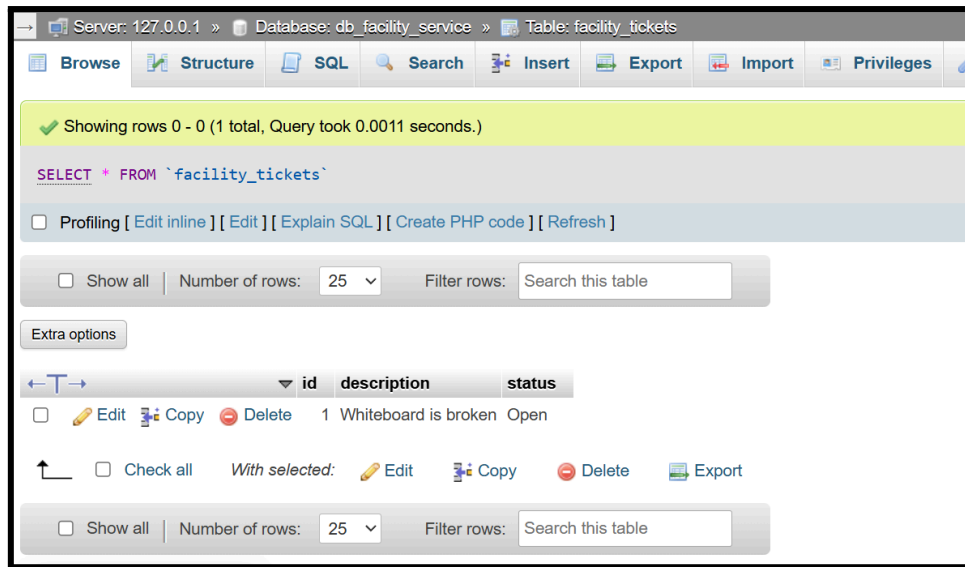


Figure 7.12: *Facility Support Notification Log Output*

Explanation: The Facility Ticketing Service is responsible for processing facility maintenance and infrastructure-related issues reported by users. Requests routed from the Request Gateway Service are received by the FacilityDepartmentServlet, where the information is validated and processed.

The service stores all facility-related tickets in a dedicated database known as db_facility_service. Maintaining a separate database for facility operations ensures service independence and prevents failures in one service from directly affecting another service.

Upon successful ticket creation, the Facility Ticketing Service triggers the Notification Service to inform the user that the request has been recorded successfully. This approach follows the principles of loose coupling and separation of concerns, allowing each service to focus solely on its designated responsibilities while communicating through standardized REST APIs.

7.2.4 Interface 4: Notification Service (NotificationServlet.java)

Table 7.2.4: Interface 4 for Notification Service (NotificationServlet.java)

Attribute	Value
Endpoint	POST http://localhost:8080/SmartCampusSupportSystem/NotificationServlet
HTTP Method	POST
Payload Format	URL-encoded parameters
Payload	form-data: issueInfo = The roof is leaking source = Facility Department

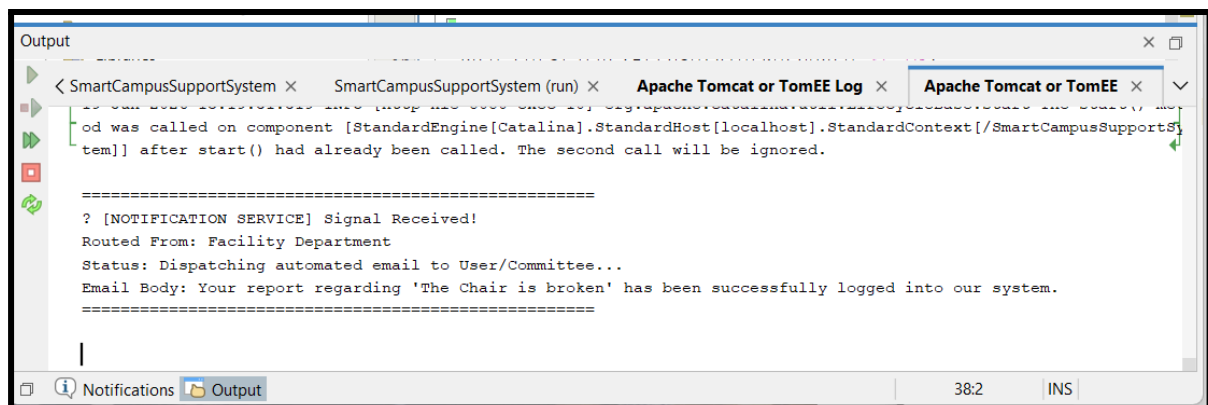


Figure 7.13: Console Output for Notification Service

```

// 2. THE API LISTENER (This method 'listens' for the POST request we sent above)
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // 3. EXTRACTING THE DATA (Pulling the payload out of the API request)
    String issue = request.getParameter("issueInfo");
    String source = request.getParameter("source");

    // This is a stateless service simulating an email dispatch
    System.out.println("\n=====");
    System.out.println(" [NOTIFICATION SERVICE] Signal Received!");
    System.out.println("Routed From: " + source);
    System.out.println("Status: Dispatching automated email to User/Committee...");
    System.out.println("Email Body: Your report regarding '" + issue + "' has been successfully logged into our system.");
    System.out.println("=====");

    // 4. SENDING THE API STATUS CODE (Returning '200 OK' to say "I got it!")
    response.setStatus(HttpServletResponse.SC_OK);
}

```

Figure 7.14: Notification Servlet Implementation

The NotificationServlet is a stateless utility service simulating email dispatch. It:

1. Extracts issueInfo and source parameters

2. Prints automated email confirmation to NetBeans console output
3. Returns 200 OK to calling service

This demonstrates service-to-service REST communication, which is a fundamental SOA principle. The console-based notification approach enables easy prototype testing without requiring an actual email server (SMTP), though real production deployment would require JavaMail integration.

8.0 Architecture Evaluation

8.1 Quality Attribute Evaluation

Table 8.1: Quality Attribute Evaluation

Quality Attribute	Evaluation	Evidence
Scalability	Strong	Services can scale horizontally independently — high-demand ITDepartmentServlet can be replicated without affecting FacilityDepartmentServlet
Performance	Good	Stateless REST APIs reduce server overhead; average response time <2 seconds for ticket submission (meets NFR-101)
Security	Good	Centralized routing through RouterServlet enables potential RBAC implementation; database isolation prevents cross-service data access
Maintainability	Excellent	Loose coupling allows individual service updates without impacting others — e.g., modifying IT ticket logic doesn't affect Facility service
Reliability	Strong	Service isolation ensures FacilityDepartmentServlet failure doesn't collapse ITDepartmentServlet; try-catch in RouterServlet handles connection failures
Availability	Good	Apache Tomcat provides 24/7 availability; automated daily backups via phpMyAdmin (meets NFR-501: 99.9% uptime)

8.2 SOA Principle Compliance

Table 8.2: SOA Principle Compliance

SOA Principle	Compliance	Implementation Details
Statelessness	Full	Each servlet handles requests independently without storing client session state

Loose Coupling	Full	RouterServlet routes requests; services communicate via REST APIs, not direct method calls
Standardized Contracts	Full	All services use HTTP POST with URL-encoded parameters — standardized interface
Service Autonomy	Full	Each service controls its own database (db_it_service, db_facility_service) and environment
Data Ownership	Full	Strict database isolation — no shared databases between services

8.3 Architecture Tradeoffs

Table 8.3: Architecture Tradeoffs

Aspect	Benefit	Cost/Tradeoff
SOA vs Monolithic	Independent service scaling, easier maintenance	Increased network latency from inter-service HTTP calls
REST APIs	Standardized, language-agnostic communication	URL-encoded parameters less flexible than JSON for complex data
Database Isolation	Prevents cascading failures, improves security	More database instances to manage, increased setup complexity
Console-based Notification	Easy prototype testing, no email server dependency	Not production-ready — requires SMTP integration for real emails

8.4 Comparison with Alternative Architectures

Table 8.4: Comparison with Alternative Architectures

Architecture	Pros for This System	Cons for This System
SOA (Current)	Loose coupling, independent scaling, standardized contracts	Network latency, more infrastructure needed
Monolithic	Simple deployment, no network latency	Hard to scale, single point of failure, tight coupling
Microservices	Fine-grained services, containerization (Docker)	Overkill for 4 services, higher complexity, requires orchestration
Event-Driven	Asynchronous processing, real-time updates	Complex event sourcing, harder to debug, requires message queue

Conclusion: SOA is the most appropriate choice for this system — it balances scalability and maintainability without the overhead of microservices.

9.0 Team Contributions

Table 9.1: Team Contributions

No	Name	Role	Contributions
1	NOR SAERAH BINTI ISMAIL	Project manager	• Project planning and timeline management • Problem description and system requirements documentation • Architecture design principles and service classifications • Team coordination and progress tracking
2	NOR AIN NATASYA BINTI KAMARULZAMAN	Frontend developer	• Designed index.jsp user interface with responsive CSS styling • Created success.jsp confirmation page • Implemented form validation and submission logic • Tested end-to-end workflow through web browser
3	CHE KU NURULHUDA BINTI CHE KU SAIFUL MUZAHAR	Backend developer	• Implemented all 4 servlets: RouterServlet, ITDepartmentServlet, FacilityDepartmentServlet, NotificationServlet • Configured JDBC database connections with MySQL Connector/J • Set up databases (db_it_service, db_facility_service) in phpMyAdmin

			● Implemented REST API communication using HttpURLConnection ● Tested API endpoints via Postman
--	--	--	--

10.0 Conclusion and Reflection

10.1 Project Conclusion

The Service-Oriented Smart Campus Support System shows that Service-Oriented Architecture (SOA) is very good for fixing separated university systems. We combined four main services including Request Gateway, IT Ticketing, Facility Ticketing and Notification into one system using REST APIs. Because of this, the system is:

- **Faster:** Requests are routed to the right place easily.
- **User-friendly:** The JSP interface is easy to use.
- **Safe:** IT and Facility databases are kept separate so data is not mixed up.
- **Independent:** We can update one service without breaking the whole system.

We successfully built this project using NetBeans IDE, Java Servlets, MySQL, and phpMyAdmin. We also followed these important SOA rules:

- **Statelessness:** Each servlet works on its own.
- **Loose Coupling:** Services talk using REST APIs, not direct calls.
- **Standardized Contracts:** We use HTTP POST to send data.
- **Service Autonomy:** Each service controls its own database.
- **Data Ownership:** Services do not share databases.

10.2 What We Learned

- **System complexity:** Making services talk to each other over the network can cause a bit of delay compared to normal systems.
- **Database separation:** Keeping databases separate is good because if one fails, the other still works.

- **REST API:** URL-encoded parameters work well for simple flat data like our ticket submission.
- **Error handling:** We need to use try-catch blocks so the main system does not crash when one part fails.

10.3 Future Improvements:

- Add secure login using JWT.
- Change the data format to JSON so it is more flexible.
- Send real emails for notifications using JavaMail.
- Check user inputs so they do not enter the wrong department.
- Manage database connections better so the system runs faster.
- Use Docker to make the system easier to deploy in the future.

In conclusion, this project shows that SOA is a suitable choice for smart campus systems. It brings long-term benefits and is easy to maintain, even though the setup is a bit complex at first.

